

1. What is Inference?

- Inference is the process of deriving new sentences (conclusions) from a knowledge base (KB).
- Uses the syntactic form of sentences (not the meaning directly) to produce entailed sentences.
- In AI, inference allows an agent to answer queries, derive new facts, and reason about the world.

Key Idea: Inference is a mechanical process that produces new sentences that follow from what is already known.

Example: From KB, infer whether King(John) holds.

3. Soundness

- An inference procedure is **sound** if it only derives sentences that are entailed by the KB.
- Formally: if $KB \models \alpha$, then $KB \models \alpha$.
- A sound procedure never produces false conclusions.
- Soundness guarantees correctness of the answers produced by inference.

Example: If an inference algorithm derives King(John), and it is sound, then King(John) must be true in all models where the KB is true.

5. Soundness and Completeness Together

- A good inference procedure is both sound and complete.
- Soundness: no incorrect conclusions.
- Completeness: no correct conclusions are missed.
- In practice, many inference procedures for FOL are sound and complete (e.g., resolution).
- This gives us a solid foundation for automated reasoning in first-order logic.

Key Takeaway: Soundness gives trust in the answers, completeness gives the ability to find all answers.

7. Why Inference in FOL is Harder than in Propositional Logic

- FOL has variables, functions, and quantifiers, leading to infinitely many possible ground instances.
- There are many ways to instantiate variables (unification).
- Must handle quantifiers ($\forall, \exists$) and their scope.
- Need to reason about objects, their properties, and relations.
- Inference rules are more complex than in propositional logic.
- The search space can be very large (sometimes infinite).

Key Idea: FOL is more expressive, but this extra expressive power makes inference significantly more challenging than in propositional logic.

2. Entailment vs. Inference

Entailment ($\models$)

- A semantic relation between sentences and models.
- $\alpha \models \beta$ means β is true in every model where α is true.
- It is a property of the meaning of sentences.
- Entailment is defined independent of any algorithm.

Inference ($\vdash$)

- A syntactic process (algorithm) that derives new sentences from a KB.
- Takes sentences as input and produces new sentences.
- It aims to be sound and complete with respect to $\models$.
- Inference is an algorithmic approximation to entailment.

Relationship: If $KB \models \alpha$, then ideally $KB \vdash \alpha$. A good inference procedure is both sound and complete, so $KB \vdash \alpha$ if and only if $KB \models \alpha$.

4. Completeness

- An inference procedure is **complete** if it can derive every sentence that is entailed by the KB.
- Formally: if $KB \models \alpha$, then $KB \vdash \alpha$.
- A complete procedure will not miss any valid conclusions.
- Completeness guarantees that all true consequences can in principle be derived.

Example: If $KB \models \text{Loves}(\text{John}, \text{Mary})$, and the inference procedure is complete, then it will be able to derive Loves(John, Mary).

6. Goal of Theorem Proving

- Given a knowledge base KB and a query sentence α , the goal is to determine whether $KB \models \alpha$.
- A theorem prover is an algorithm that attempts to derive α from KB using a set of inference rules.
- If successful, it outputs a proof (a sequence of inferred sentences).
- If $KB \wedge \neg \alpha$ leads to a contradiction, then α is entailed by KB (proof by refutation).

Example Query:

KB: 1. King(John)
2. $\forall x (\text{King}(x) \rightarrow \text{Person}(x))$

Query: Person(John)

Goal: Derive Person(John) from KB.

8. Comparison: Propositional vs. First-Order Logic

Aspect	Propositional Logic	First-Order Logic
Basic unit	Atomic proposition (P, Q, ...)	Predicates, terms (P(x), f(x), ...)
Objects	No explicit objects	Explicit objects, constants, variables, functions
Expressiveness	Limited	Much more expressive
Quantifiers	Not available	$\forall$ (for all), $\exists$ (there exists)
Domain	Implicit (finite set of propositions)	Explicit (possibly infinite domain of objects)
Inference difficulty	Relatively simpler	More complex (unification, quantifiers, many instances)

“Inference allows an agent to derive new knowledge from what it already knows. In first-order logic, inference is more challenging but also much more powerful.”

1. Substitutions

- A substitution (θ) maps variables to terms.
- Written as $\{x/t, y/s, \dots\}$ or $\{x \mapsto t, y \mapsto s\}$.
- Applying a substitution to an expression replaces variables with the corresponding terms.
- Can be applied to terms, atoms, or sentences.

Key Idea: Substitutions allow us to make two expressions identical by replacing variables with appropriate terms.

2. Examples of Substitutions

Example 1: $\theta = \{x/\text{John}, y/f(\text{Mary})\}$

- $\theta(\text{Knows}(x, y)) = \text{Knows}(\text{John}, f(\text{Mary}))$
- $\theta(\text{Father}(x, z)) = \text{Father}(\text{John}, z)$
- $\theta(P(x) \wedge Q(y)) = P(\text{John}) \wedge Q(f(\text{Mary}))$

Example 2: $\theta = \{x/a, y/g(z)\}$

- $\theta(R(x, y, z)) = R(a, g(z), z)$
- $\theta(\neg P(y)) = \neg P(g(z))$
- $\theta(\forall x \text{ Knows}(x, y)) = \forall x \text{ Knows}(x, g(z))$
(bound variables are not substituted)

3. Unification

- Unification finds a substitution that makes two expressions syntactically identical.
- Two expressions unify if there exists a substitution θ such that $\theta(E_1) = \theta(E_2)$.
- Used to match a query with a fact, to resolve literals, and in many inference algorithms.
- If no such substitution exists, the expressions do not unify.

4. Unification Examples

Example 1: $\text{Unify}(\text{Knows}(\text{John}, x), \text{Knows}(\text{John}, \text{Jane}))$

- Match the function/predicate: Knows with Knows ✓
- Match first argument: John with John ✓
- Match second argument: x with Jane
→ substitution $\{x/\text{Jane}\}$
- Result: $\theta = \{x/\text{Jane}\}$

Thus, $\text{Knows}(\text{John}, x)$ and $\text{Knows}(\text{John}, \text{Jane})$ unify with $\theta = \{x/\text{Jane}\}$.

Example 2: $\text{Unify}(P(x, f(y)), P(a, f(b)))$

- Match predicate: P with P ✓
- Match first argument: x with a
→ $\{x/a\}$
- Match second argument: $f(y)$ with $f(b)$
→ y with b → $\{y/b\}$
- Combined substitution: $\theta = \{x/a, y/b\}$

Result: $P(x, f(y))$ and $P(a, f(b))$ unify with $\theta = \{x/a, y/b\}$.

Example 3: Not Unifiable

$\text{Unify}(\text{Knows}(\text{John}, x), \text{Knows}(\text{Mary}, x))$

- Match predicate: Knows with Knows ✓
- Match first argument: John with Mary
→ different constants (cannot be unified)

Result: No substitution can make these identical, so they do not unify.

5. Occurs Check

- A variable cannot be unified with a term that contains the same variable (would create an infinite term).
- This is known as the occurs check.

Example: $\text{Unify}(x, f(x))$

- Would require substitution $\{x/f(x)\}$
- This makes x occur inside $f(x)$.
- Fails the occurs check → not unifiable.

6. Most General Unifier (MGU)

- A most general unifier (MGU) is a unifier that is more general than any other unifier.
- If θ is an MGU of E_1 and E_2 , then for any other unifier σ , there exists a substitution γ such that $\sigma = \gamma \circ \theta$ (σ is an instance of θ).
- The MGU makes the minimal necessary substitutions, leaving variables as general as possible.

Key Idea: The MGU captures the most general way to make two expressions identical.

7. MGU Examples

Example 1: $\text{Unify}(P(x, y), P(a, b))$

- MGU: $\theta = \{x/a, y/b\}$
- Any other unifier is an instance of this one (e.g., $\{x/a, y/b, z/c\}$).

Example 2: $\text{Unify}(P(x, x), P(y, a))$

- From first and second arguments: x must match both y and a .
- MGU: $\theta = \{x/a, y/a\}$
- (More specific unifiers like $\{x/a, y/a, z/b\}$ are instances of this MGU.)

8. Comparing Unifiers

Unifier	Substitution	Is an MGU?	Notes
θ_1	$\{x/a\}$	Yes	Most general unifier for $P(x), P(a)$.
θ_2	$\{x/a, y/b\}$	No	More specific (instance of θ_1).
θ_3	$\{x/f(y)\}$	Yes	MGU for $P(x), P(f(y))$.
θ_4	$\{x/f(a)\}$	No	More specific (instance of θ_3).

9. Lifting Propositional Inference to FOL

- Many propositional inference rules can be extended to first-order logic using unification.
- To apply a rule, we first unify the relevant literals to find the substitution, and then apply the rule with that substitution.
- Example: Modus Ponens for FOL

From: $P(x)$ and $P(a) \rightarrow Q(a)$
Unify $P(x)$ with $P(a)$: $\theta = \{x/a\}$
Apply substitution to the rule: $Q(a)$
Therefore, infer: $Q(a)$.

10. Summary and Takeaways

- Substitutions map variables to terms.
- Unification finds a substitution that makes two expressions identical.
- The most general unifier (MGU) is the simplest, most general unifier.
- Occurs check prevents infinite terms.
- Unification is central to first-order inference algorithms (e.g., resolution).
- Many propositional inference rules can be lifted to FOL using unification.

Key Takeaway: Unification is the key mechanism that allows us to apply general inference rules to first-order logic expressions. It identifies how to make two expressions match, and the most general unifier captures the minimal, most general substitution needed.

1. Universal Instantiation (UI)

- From a universally quantified sentence, we can infer a ground instance by substituting a term for the variable.
- Allows us to use general knowledge about all objects for a specific object.

Rule (UI): From $\forall x P(x)$, infer $P(t)$ for any term t (that is free for x).

Example: $\frac{\forall x \text{ Human}(x)}{\therefore \text{Human}(\text{John})}$ {UI with $t = \text{John}$ }

2. Existential Instantiation (EI)

- From an existentially quantified sentence, we can infer a new constant to refer to a specific (but unknown) object that satisfies the property.
- The new constant is called a skolem constant (a name for an arbitrary object).

Rule (EI): From $\exists x P(x)$, infer $P(c)$ for a new constant c (not used elsewhere).

Example: $\frac{\exists x \text{ Cat}(x)}{\therefore \text{Cat}(c)}$ {EI with new constant c }

3. Generalized Modus Ponens (GMP)

- A rule of inference for first-order logic using unification.
- If a rule's antecedent matches known facts (possibly after substitution), we can infer a new conclusion.

Rule (GMP): From $(P_1 \wedge \dots \wedge P_n \Rightarrow Q)$ and facts $P_1', \dots, P_n'$ that unify with $P_1, \dots, P_n$, infer Q with the same substitution θ .

Example: $\frac{\forall x (\text{Human}(x) \Rightarrow \text{Mortal}(x))}{\text{Human}(\text{Socrates})} \frac{\text{Human}(\text{Socrates})}{\therefore \text{Mortal}(\text{Socrates})}$ {GMP with $\theta = \{x/\text{Socrates}\}$ }

4. Forward Chaining

- A data-driven inference algorithm.
- Start with known facts and repeatedly apply inference rules to generate new facts until no new facts can be inferred.
- Often used with Horn clauses.
- Terminates for finite sets of ground facts and rules.

Key Idea: Repeatedly apply GMP to rules whose antecedents are satisfied by known facts, adding new facts to the knowledge base.

5. Forward Chaining Example

Knowledge base:

- $\text{Parent}(x, y) \Rightarrow \text{Ancestor}(x, y)$
- $\text{Parent}(x, y) \Rightarrow \text{Ancestor}(x, z)$ if $\text{Ancestor}(y, z)$
- $\text{Parent}(\text{John}, \text{Mary})$
- $\text{Parent}(\text{Mary}, \text{Bob})$

Inference trace (forward chaining):

1.	$\text{Parent}(\text{John}, \text{Mary}), \text{Parent}(\text{Mary}, \text{Bob})$	(given)
2.	$\text{Ancestor}(\text{John}, \text{Mary})$	(from 1, rule 1)
3.	$\text{Ancestor}(\text{Mary}, \text{Bob})$	(from 1, rule 1)
4.	$\text{Ancestor}(\text{John}, \text{Bob})$	(from 2 and 3, rule 2)
5.	(no new facts)	(terminate)

6. Horn Clauses

- A Horn clause is a disjunction of literals with at most one positive literal.
- Equivalent form: a rule with a conjunction of positive literals implying a single positive literal.
- Widely used in logic programming and forward chaining.

Form: $A_1 \wedge \dots \wedge A_n \Rightarrow B$
($A_1, \dots, A_n$ are positive literals)

Example (Horn clause):

$\text{Parent}(x, y) \wedge \text{Ancestor}(y, z) \Rightarrow \text{Ancestor}(x, z)$

7. Rule-Based Inference

- A set of Horn clauses can be viewed as a rule-based system.
- Rules encode domain knowledge; facts represent known information.
- Inference is performed by repeatedly applying rules to derive new facts (e.g., using forward chaining).
- Common in expert systems and knowledge representation.

Components:

- Facts:** ground atomic sentences (in the KB).
- Rules:** Horn clauses (if antecedents are true, infer consequent).
- Inference engine:** applies rules (e.g., forward chaining).

Example: Rule-based system for family relations

R1: $\text{Parent}(x, y) \Rightarrow \text{Ancestor}(x, y)$
R2: $\text{Parent}(x, y) \wedge \text{Ancestor}(y, z) \Rightarrow \text{Ancestor}(x, z)$
F1: $\text{Parent}(\text{John}, \text{Mary})$
F2: $\text{Parent}(\text{Mary}, \text{Bob})$

Query: $\text{Ancestor}(\text{John}, \text{Bob}) ? \rightarrow \text{Yes}$ (inferred by forward chaining)

8. Forward Chaining Algorithm (for Horn Clauses)

Algorithm (Forward Chaining)

- Initialize the set of inferred facts with the known facts.
- Repeat until no new facts are inferred:
 - For each rule, check if all antecedents are satisfied by known facts (using unification).
 - If so, infer the consequent (with the same substitution).
 - Add the new fact to the set (if not already present).
 - Mark that a new fact was inferred.
- Return the set of inferred facts.

The algorithm is sound and complete for definite (Horn) clauses.

9. Key Takeaways

- Universal instantiation allows us to use general rules for a specific object.
- Existential instantiation introduces a new constant for an unknown object.
- Generalized modus ponens uses unification to infer new facts.
- Forward chaining is a data-driven inference algorithm.
- Horn clauses have at most one positive literal, written as rules.
- Rule-based inference applies Horn clauses to derive new facts.
- Forward chaining is sound and complete for definite clauses.

“From general knowledge to specific facts — inference lets us derive new truths.”

— AIMA (Ch. 9)

1. What is Backward Chaining?

- A goal-directed inference algorithm for first-order logic.
- Starts with a query (goal) and works backwards to find supporting facts.
- Uses rules to reduce the goal to subgoals, recursively, until they are known to be true.
- Systematic, depth-first search with unification.

Key Idea: Backward chaining proves a query by recursively reducing it to subgoals that match the antecedents of rules.

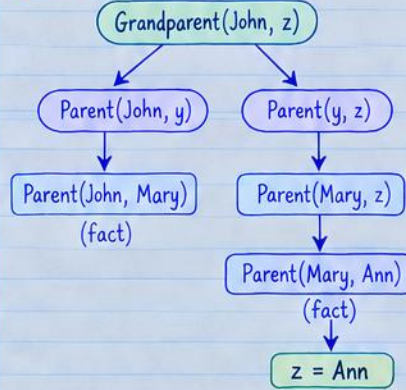
Example: Prove $\text{Grandparent}(\text{John}, z)$.

2. Backward Chaining Example

Rules:

1. $\text{Parent}(x, y) \wedge \text{Parent}(y, z) \Rightarrow \text{Grandparent}(x, z)$
2. $\text{Parent}(\text{John}, \text{Mary})$
3. $\text{Parent}(\text{Mary}, \text{Ann})$

Query: $\text{Grandparent}(\text{John}, z)$



3. Goal-Directed Inference

- Backward chaining is goal-directed: focuses only on rules relevant to the query.
- Uses unification to match the goal with the head of a rule and generate subgoals from the rule's antecedents.
- Recursively prove subgoals, using facts or further rules.
- Backtracks if a subgoal fails (depth-first).
- Can be seen as a form of AND-OR search over a goal tree.

Key Idea: Goal-directed inference tries to prove a query by reducing it to subgoals, exploring only relevant parts of the knowledge base.

Characteristics:

- Starts from the query (goal).
- Uses unification and recursive reasoning.
- More efficient than forward chaining when the query is specific.
- May not terminate if there are recursive rules or infinite branches.

4. Recursive Reasoning

- Backward chaining naturally handles recursive rules.
- Rules can refer to the same predicate in their body (directly or indirectly).
- Can be used to prove properties over arbitrarily long structures (e.g., ancestor).

Example: Ancestor relation (recursive rule)

1. $\text{Parent}(x, y) \Rightarrow \text{Ancestor}(x, y)$
2. $\text{Parent}(x, y) \wedge \text{Ancestor}(y, z) \Rightarrow \text{Ancestor}(x, z)$

Query: $\text{Ancestor}(\text{John}, \text{Ann})$

Backward chaining generates subgoals like $\text{Parent}(\text{John}, y)$ and $\text{Ancestor}(y, \text{Ann})$ recursively until facts are found.

5. Resolution in First-Order Logic

- Resolution is a complete inference rule for first-order logic (refutation method).
- Works with clauses (disjunctions of literals) in conjunctive normal form (CNF).
- To prove a query α , we show that $\text{KB} \wedge \neg \alpha$ is unsatisfiable (leads to contradiction).
- Repeatedly apply the resolution rule to derive new clauses until the empty clause is produced.

Key Idea: Resolution proves a query by showing that $\text{KB} \wedge \neg \alpha$ leads to a contradiction (empty clause).

6. Conjunctive Normal Form (CNF)

- Resolution works with clauses in CNF.
- A formula is in CNF if it is a conjunction of disjunctions of literals.
- Conversion to CNF (basics):

1. Eliminate biconditionals ($\leftrightarrow$).
2. Eliminate implications ($\Rightarrow$).
3. Move negations inward (De Morgan's laws).
4. Rename variables (standardize apart).
5. Skolemize (eliminate existential quantifiers).
6. Drop universal quantifiers (clauses are implicitly universally quantified).
7. Distribute $\vee$ over $\wedge$.

Example: $\exists x (\text{P}(x) \wedge \text{Q}(x)) \rightarrow \text{P}(a) \wedge \text{Q}(a)$
(after skolemization)

7. Skolemization

- Eliminates existential quantifiers by introducing new constant or function symbols (Skolem symbols).
- Preserves satisfiability (but not logical equivalence).
- If no universally quantified variables are in scope, use a new constant.
- If universally quantified variables are in scope, use a new function of those variables.

Examples:

- $\exists x \text{P}(x) \rightarrow \text{P}(a)$ (a is a new constant)
- $\forall y \exists x \text{P}(x, y) \rightarrow \text{P}(f(y), y)$
(f is a new function symbol)

8. Resolution Rule (FOL)

- Given two clauses with complementary literals, resolve them to produce a new clause (resolvent) using unification.
- If $C_1 = (L \vee A)$ and $C_2 = (\neg L' \vee B)$, where L and L' unify with substitution θ , then the resolvent is $(A \vee B)\theta$.

Example:

$C_1: \text{P}(x) \vee \text{Q}(x)$

$C_2: \neg \text{P}(a) \vee \text{R}(a)$

Unify $\text{P}(x)$ with $\text{P}(a): \{x/a\}$

Resolvent: $\text{Q}(a) \vee \text{R}(a)$

9. Proof by Contradiction (Refutation)

- To prove $\text{KB} \models \alpha$:

1. Add the negation of the query to KB: $\text{KB} \wedge \neg \alpha$.
2. Convert all sentences to CNF (include Skolemization).
3. Repeatedly apply resolution to derive new clauses.
4. If the empty clause ($\square$) is derived, then $\text{KB} \models \alpha$.
5. If no new clauses can be generated, the proof fails (query not entailed).

Example (outline): Prove $\text{KB} \models \alpha$

1. Start with $\text{KB} \wedge \neg \alpha$.
2. Convert to CNF (Skolemize, etc.).
3. Apply resolution until $\square$ is derived.
4. If $\square$ is derived, α is entailed.

1. Limits of Inference

- FOL is more expressive than propositional logic, but inference has fundamental limits.
- Not all useful knowledge can be captured in FOL (e.g., uncertainty, probability, time, actions, defaults).
- Some statements are true in the real world but cannot be proven from given axioms.
- Requires additional formalisms (e.g., temporal logic, probabilistic models, non-monotonic logic).

Key Idea: FOL is powerful, but it does not capture everything about the real world.

2. Semi-Decidability of FOL

- FOL entailment is semi-decidable.
- If a sentence α is entailed by KB, a sound inference procedure will eventually find a proof.
- If α is not entailed, the procedure may run forever (no proof will be found).
- There is no general algorithm that always halts with a yes or no answer for arbitrary FOL sentences.
- This is due to the undecidability of first-order logic.

Semi-decidable means we can recognize entailed sentences, but we may never halt on non-entailed sentences.

3. Efficiency Issues

- FOL inference can be computationally expensive.
- The search space can be very large due to:
 - Large number of possible substitutions.
 - Many possible inferences.
 - Deep proof trees.
- Unification, variable bindings and clause generation add overhead.
- Inference is often exponential or worse in the size of the knowledge base.
- Practical systems use heuristics, indexing, and restrictions (e.g., Horn clauses) to improve efficiency.

4. When to Use Forward vs. Backward Chaining

Forward Chaining (Data-Driven)

- Start from known facts and infer new facts.
- Useful when we want to derive all possible consequences.
- Good when there are many queries or we want to find all conclusions.
- Often used in production systems and rule-based systems.

Example situations:

- Monitoring, planning, simulation.
- When new data arrives continuously.
- When we want all consequences.

Backward Chaining (Goal-Driven)

- Start from a query and work backwards to find supporting facts.
- Useful when we have a specific goal or query to prove.
- Only explores relevant parts of the knowledge base.
- Often used in logic programming (e.g., Prolog).

Example situations:

- Answering specific queries.
- Diagnosis, theorem proving.
- When the number of possible goals is small.

5. Comparison of Inference Methods

Method	Idea	Advantages	Disadvantages	Typical Use
Forward Chaining	Data-driven (derive new facts)	Finds all consequences, useful for many queries	Can generate many irrelevant facts (inefficient)	Rule-based systems, production systems
Backward Chaining	Goal-driven (start from query)	Focuses on relevant facts, more efficient for specific goals	May repeat work for multiple queries, can get stuck with loops	Logic programming (e.g., Prolog)
Resolution	Refutation (using clauses)	Complete for FOL, well-studied	Can be inefficient, large number of clauses	General purpose inference
Unification	Find substitution to make terms equal	Essential for FOL inference, efficient algorithms exist	May fail when terms do not unify	Used in most FOL inference methods

6. Common Mistakes

- Assuming FOL inference always halts.
- Confusing forward and backward chaining.
- Forgetting to apply unification correctly.
- Not standardizing variables apart before applying inference rules.
- Ignoring that inference can generate many irrelevant facts.
- Assuming FOL can represent uncertainty or default knowledge.
- Forgetting that a lack of proof does not mean the sentence is false.

7. Exam Tips

- Be clear about the inference method being used (forward, backward, resolution).
- Show unification steps explicitly.
- Standardize variables apart to avoid name clashes.
- Draw inference trees or proof steps neatly.
- Remember that FOL inference is semi-decidable (may not halt).
- For conceptual questions, explain when each method is appropriate.
- Practice small examples of forward and backward chaining.
- Understand the advantages and limitations of each method.

8. Key Takeaways

- FOL inference has fundamental limits and is semi-decidable.
- Inference can be computationally expensive due to large search spaces.
- Use forward chaining when we want all consequences or have many queries.
- Use backward chaining when we have a specific goal or query.
- Different inference methods have different strengths and weaknesses.
- Be aware of common mistakes such as variable handling and incorrect unification.
- Inference is a powerful tool, but FOL does not capture everything about the real world.
- In practice, heuristics and restrictions are used to improve efficiency.

9. Summary Table

Aspect	Forward Chaining	Backward Chaining	Resolution (Refutation)	Unification
Search direction	From facts to conclusions	From goal to facts	Refute negation of goal	Find substitutions
When useful	Many queries, all results	Specific query	General purpose	Needed in all FOL methods
Explores	Derives new facts (data-driven)	Relevant subgoals (goal-driven)	Clause space	Terms and variables
Completeness	Yes (for definite clauses)	Yes (for definite clauses)	Yes (for FOL)	Complete (for terms)
Efficiency	Can be inefficient (many facts)	More efficient for specific goals	Often expensive	Usually efficient
Typical use	Production systems	Logic programming (Prolog)	Theorem proving	

“Inference in first-order logic is powerful but has limits, so we use the right method for the right goal and practical techniques to manage the complexity.”